

Optimizing Data Analytics Workflows via Human: AI Collaboration

AI Tool Used: ChatGPT (GPT-4o)

AI Use Declaration: This task was completed using ChatGPT (GPT-4o) as an interactive coding assistant. All architectural design, error-handling validation, and final revisions were performed by the human author.

1. Task Description

Overview of the Task

The objective of this task is to build an automated data processing and analytics pipeline in Python. The task involves taking a raw, unverified corporate dataset (`sales_data.csv`), identifying missing values, cleaning them, aggregating sales by product, and producing a clear bar chart for decision-making purposes.

Why This Task Was Chosen

Data cleaning and descriptive analytics are core skills in data engineering coursework. In practice, raw data is rarely clean, and manually scanning for missing values, writing aggregation logic, and debugging plotting code can be repetitive and time-consuming. This task was chosen to evaluate how an AI assistant can help shift effort from low-level syntax writing toward higher-level review, validation, and design decisions, while keeping the human fully responsible for correctness and quality.

2. AI Tool

Tool Selected

The AI tool selected for this task is ChatGPT, using the GPT-4o model.

Why This Tool Was Selected

ChatGPT was chosen because of its strong conversational reasoning, ability to retain context across follow-up prompts, and broad familiarity with common Python data libraries such as:

- Pandas – for loading and manipulating structured tabular data
- Matplotlib – for generating charts and visualizations

This made it well suited to a short, iterative workflow where an initial rough request is refined into a more structured and production-ready script.

3. AI Workflow

Phase A – Original Human Draft

The workflow began with an informal, unstructured request written by the human author, describing the task in plain language:

“
I
h
a
v
e
a
m
e
s
s
y
C
S
V
f
i
l
e
n
a
m
e
d
,
s
a
l
e
s
—
d
a
t
a
.
c
s
v
,
.
I
t
c
o
n
t

*a
i
n
s
t
h
e
c
o
l
u
m
n
s
,
D
a
t
e
,
,
P
r
o
d
u
c
t
,
,
S
a
l
e
s
,
,
a
n
d
,
R
e
g
i
o
n
,*

.
S
o
m
e
r
o
w
s
h
a
v
e
m
i
s
s
i
n
g
v
a
l
u
e
s
i
n
t
h
e
s
a
l
e
s
c
o
l
u
m
n
. I
w
a
n
t
a
s

c
r
i
p
t
t
o
f
i
l
l
t
h
o
s
e
e
m
p
t
y
s
p
o
t
s
w
i
t
h
z
e
r
o
,
g
r
o
u
p
t
h
e
t
o
t
a
l
s
a

*l
e
s
b
y
p
r
o
d
u
c
t
,
a
n
d
o
u
t
p
u
t
a
c
l
e
a
n
b
a
r
g
r
a
p
h
s
o
m
y
t
e
a
m
c
a
n
s
e
e*

w
h
i
c
h
p
r
o
d
u
c
t
s
a
r
e
p
e
r
f
o
r
m
i
n
g
b
e
s
t
.
”

Phase B – Refined Prompt Used

To get a more complete and well-structured response, the request was rewritten as a clearer, step-by-step prompt:

A
c
t
a
s
a
d
a
t
a
e
n
g
i
n

```
e
e
r
.
I
a
m
p
r
o
c
e
s
s
i
n
g
a
r
a
w
b
u
s
i
n
e
s
s
d
a
t
a
s
e
t
(
'
s
a
l
e
s
_
d
a
t
a
.
c
s
v
'
)
w
i
t
h
c
o
l
u
m
```

n
s
,
D
a
t
e
,
,
P
r
o
d
u
c
t
,
,
S
a
l
e
s
,
,
R
e
g
i
o
n
,
.

P
l
e
a
s
e
:
1
.
W
r
i
t
e
a
P
y
t
h
o
n
s
c
r
i
p

t
u
s
i
n
g
p
a
n
d
a
s
t
o
l
o
a
d
t
h
e
f
i
l
e
s
a
f
e
l
y
.2
.I
d
e
n
t
i
f
y
m
i
s
s
i
n
g
v
a
l
u
e
s
i
n
t
h
e
,
S
a

less, column and replace the m with 0.
3.3. Group the data by product, and compute

t
e
t
o
t
a
l
.
S
a
l
e
s
.
p
e
r
p
r
o
d
u
c
t
.
4
.
G
e
n
e
r
a
t
e
a
b
a
r
c
h
a
r
t
w
i
t
h
.
P
r
o
d
u
c
t
.
o
n
t
h
e
x

-
a
x
i
s
a
n
d
.
T
o
t
a
l
S
a
l
e
s
.
o
n
t
h
e
y
-
a
x
i
s
.
5
.
A
d
d
s
h
o
r
t
i
n
l
i
n
e
c
o
m
m
e
n
t
s
e
x
p
l
a
i

```
n  
i  
n  
g  
e  
a  
c  
h  
s  
t  
e  
p  
.
```

Phase C – AI-Generated Output

ChatGPT returned the following functional script:

```
i  
m  
p  
o  
r  
t  
p  
a  
n  
d  
a  
s  
a  
s  
p  
d  
i  
m  
p  
o  
r  
t  
m  
a  
t  
p  
l  
o  
t  
l  
i  
b  
.  
p  
y  
p  
l  
o  
t  
a  
s  
p
```

```
1  
t
```

```
#  
Step  
1:  
Load  
the  
data  
set  
from  
pd.  
read_  
csv(  
'sales_  
data.  
csv',  
)
```

```
#  
Step  
2
```

```
: Handle missing values in the Sales column of df['Sales', ] = df['Sales', ].fill
```

lna(
0)

Step
3:
Group
by
Product
and
sum
the
sales
product
—
sales
=
df.
group

```
p
b
y
(
,
P
r
o
d
u
c
t
,
)
[
,
S
a
l
e
s
,
]
.
s
u
m
(
)
.
r
e
s
e
t
_
i
n
d
e
x
(
)

#
S
t
e
p
4
:
C
r
e
a
t
e
t
h
e
b
a
r
```

```
c
h
a
r
t
p
l
t
.
f
i
g
u
r
e
(
f
i
g
s
i
z
e
=
(
1
0
,
6
)
)
p
l
t
.
b
a
r
(
p
r
o
d
u
c
t
_
s
a
l
e
s
[
,
p
r
o
d
u
c
t
,
]
```

```
, product_sales[  
    'Sales',  
], color = 'blue',  
)  
plt.title('Total Sales by Pro
```

```
d
u
c
t
:
)
p
l
t
.
x
l
a
b
e
l
(
:
P
r
o
d
u
c
t
:
)
p
l
t
.
y
l
a
b
e
l
(
:
T
o
t
a
l
S
a
l
e
s
(
$
)
:
)
p
l
t
.
g
r
i
d
(
```

```
a
x
i
s
=
,
y
,
,
l
i
n
e
s
t
y
l
e
=
,
-
-
,
,
a
l
p
h
a
=
0
.
7
)
p
l
t
.
s
h
o
w
(
)
```

Phase D – Final Human-Edited Version

On review, the raw output was judged too fragile for real use: it had no error handling and the chart was not sorted or labeled clearly. The following changes were made by hand: added a try/except block around file loading, converted the sales column with `pd.to_numeric()` for safer type handling, sorted the aggregated results in descending order so the top-performing products stand out immediately, rotated the x-axis labels to prevent text overlap, and added data labels above each bar.

```
"
"
"
S
y
s
```

t
e
m
T
i
t
l
e
:
P
r
o
d
u
c
t
i
o
n
-
G
r
a
d
e
A
u
t
o
m
a
t
e
d
D
a
t
a
A
n
a
l
y
t
i
c
s
P
i
p
e
l
i
n
e
F
r
a
m
e
w
o
r

k : Human - AI Collaborative Engineering (Stanford HAS H3 Model) " " import

p
a
n
d
a
s
a
s
p
d
i
m
p
o
r
t
m
a
t
p
l
o
t
l
i
b
.
p
y
p
l
o
t
a
s
p
l
t
i
m
p
o
r
t
s
y
s

d
e
f
r
u
n
-
d
a
t
a
-
p
i
p

eline (filepath):
#1. Defensive Intelligence Layer (Human Modification):

```
T  
r  
y  
-  
E  
x  
c  
e  
p  
t  
)  
  
t  
r  
y  
:  
  
d  
f  
=  
p  
d  
.  
r  
e  
a  
d  
_  
c  
s  
v  
(  
f  
i  
l  
e  
_  
p  
a  
t  
h  
)  
  
p  
r  
i  
n  
t  
(  
f  
"  
[  
S  
U  
C  
C  
E  
S  
S  
]  
S  
u  
c
```

```
c
e
s
s
f
u
l
l
y
i
n
g
e
s
t
e
d
d
a
t
a
s
e
t
:
{
f
i
l
e
_
p
a
t
h
}
"
)
e
x
c
e
p
t
F
i
l
e
N
o
t
F
o
u
n
d
E
r
r
o
r
:
```

```
p r i n t ( f " [ C R I T I C A L E R R O R ] T a r g e t f i l e , { f i l e _ p a t h } , n o t f o u n d . " )  
s y s .
```

exit(1)

except
Exception as:

print(f"[SYSTEM ERROR] Unexpected read

```
f  
a  
i  
l  
u  
r  
e  
:  
{  
s  
t  
r  
(  
e  
)  
}  
"
```

```
)  
  
s  
y  
s  
.  
e  
x  
i  
t  
(  
1  
)
```

```
#  
2  
.  
D  
a  
t  
a  
C  
l  
e  
a  
n  
s  
i  
n  
g  
&  
D  
e  
t  
e  
r  
m  
i  
n  
i  
s  
t  
i  
c  
I
```

```
mputat ion  
df[  
Sales,  
]  
=pd.to  
numeric  
(df[  
Sales,  
],  
errors=  
,  
coerce  
.
```

)
.
f
i
l
l
n
a
(
Ø
)

3
.
D
o
w
n
s
t
r
e
a
m
A
g
g
r
e
g
a
t
i
o
n
(
H
u
m
a
n
M
o
d
i
f
i
c
a
t
i
o
n
:
S
o
r
t
i
n

```
g
)
p
r
o
d
u
c
t
_
s
a
l
e
s
=
(
d
f
.
g
r
o
u
p
p
y
(
'
p
r
o
d
u
c
t
'
)
[
'
s
a
l
e
s
'
]
.
s
u
m
(
)
.
s
o
r
t
_
v
```

```
values(ascending = False)
```

```
.reset_index()
```

```
# 4. Visual Analyticts & Char
```

```
t
G
e
n
e
r
a
t
i
o
n
(
H
u
m
a
n
M
o
d
i
f
i
c
a
t
i
o
n
)
```

```
f
i
g
,
a
x
=
p
l
t
.
s
u
b
p
l
o
t
s
(
f
i
g
s
i
z
e
=
(
1
2
,
```

7
)
)

b
a
r
s
=
a
x
.
b
a
r
(
p
r
o
d
u
c
t
_
s
a
l
e
s
[
,
p
r
o
d
u
c
t
.
,
]
,
p
r
o
d
u
c
t
_
s
a
l
e
s
[
,
S
a
l
e
s
.
,

```
] ,  
color = ,  
#4a90e2 ,  
 ,  
edge  
color = ,  
#1d3557 ,  
 ,  
linewidth  
= 1 . 2 )  
  
ax .  
set  
_t
```

i
t
l
e
(
'
S
t
r
a
t
e
g
i
c
p
e
r
f
o
r
m
a
n
c
e
R
e
v
i
e
w
:
R
e
v
e
n
u
e
b
y
p
r
o
d
u
c
t
'
,
f
o
n
t
s
i
z
e
=
1
5

```
, fontweight = 'bold',  
pad = 20)  
ax.set_xlabel('Product Classification')
```

```
L  
i  
n  
e  
,  
f  
o  
n  
t  
s  
i  
z  
e  
=  
1  
2  
,  
l  
a  
b  
e  
l  
p  
a  
d  
=  
1  
2  
)  
  
a  
x  
.s  
e  
t  
_y  
l  
a  
b  
e  
l  
(  
'  
T  
o  
t  
a  
l  
A  
c  
c  
u  
m  
u  
l  
a  
t  
e  
d  
R
```

```
e
v
e
n
u
e
(
U
S
D
$
)
,
f
o
n
t
s
i
z
e
=
1
2
,
l
a
b
e
l
p
a
d
=
1
2
)
```

```
p
l
t
.
x
t
i
c
k
s
(
r
o
t
a
t
i
o
n
=
3
5
,
```

```
h
a
=
,
r
i
g
h
t
,
,
f
o
n
t
s
i
z
e
=
1
0
)

p
l
t
.
y
t
i
c
k
s
(
f
o
n
t
s
i
z
e
=
1
0
)

f
o
r
b
a
r
i
n
b
a
r
s
:
```

```
height = bar.get_height()
ax.annotate(f'{height:0.2f}',
            xy=(bar
```

```
.get_x()  
+bar.get_width()  
/2, height),  
xytext=(0,5),  
textcoords='of
```

```
f
s
e
t
p
o
i
n
t
s
,
,
h
a
=
,
c
e
n
t
e
r
,
,
v
a
=
,
b
o
t
t
o
m
,
,
f
o
n
t
s
i
z
e
=
9
,
f
o
n
t
w
e
i
g
h
t
=
,
b
o
l
```

```
d.  
)
```

```
plt.tight_  
_layout(  
)
```

```
print(' [S  
U  
C  
C  
E  
S  
S ] A  
n  
a  
l  
y  
t  
i  
c  
a  
l  
c  
h  
a  
r  
t  
r  
e  
n  
d  
e  
r  
e  
d
```

```
;
)

plt.
show(

if
-
-
name
-
-
=
=
,
-
-
main
-
-
:
run
-
data
-
pipeline(
,
sales
```

4. Critical Reflection

Productivity Impact

Using ChatGPT noticeably reduced the time spent on boilerplate work. Writing the initial loading, cleaning, grouping, and plotting logic from scratch, and looking up specific Matplotlib parameters, would typically take significant focused effort. With AI assistance, a working first draft was available almost immediately, which let the human author spend most of their time reviewing, testing, and improving the script rather than typing out the initial structure.

Strengths of the AI Tool

- Produced syntactically correct, runnable Python code on the first attempt.
- Combined multiple libraries (Pandas and Matplotlib) correctly in one coherent script.
- Responded well to a more structured, step-by-step prompt compared to a vague one.

Limitations of the AI Tool

- Assumed an ideal environment: the original script did not handle a missing or corrupted file.
- Did not sort or prioritize the data in a way that would be useful for business decision-making.
- The chart formatting (labels, rotation, spacing) needed manual polishing to be presentation-ready.

Changes Made After Review

- Added a try/except block so the pipeline fails safely instead of crashing on a missing file.
- Used `pd.to_numeric(..., errors='coerce')` for more robust handling of unexpected values.
- Sorted results in descending order so the most important products are immediately visible.
- Rotated axis labels and added value labels on the bars to remove overlap and improve readability.

The Importance of Human Judgment

This exercise reinforced that AI tools are most useful as assistants rather than autonomous decision-makers. ChatGPT was effective at producing correct, working syntax quickly, but it could not anticipate real-world failure cases or understand what would actually be useful to a business audience. Those decisions – defensive error handling, sorting for insight, and presentation quality – depended entirely on the human reviewer. Treating AI output as a first draft to be checked, tested, and improved, rather than a finished answer, was essential to producing a script that could reasonably be used in practice.

End of report.